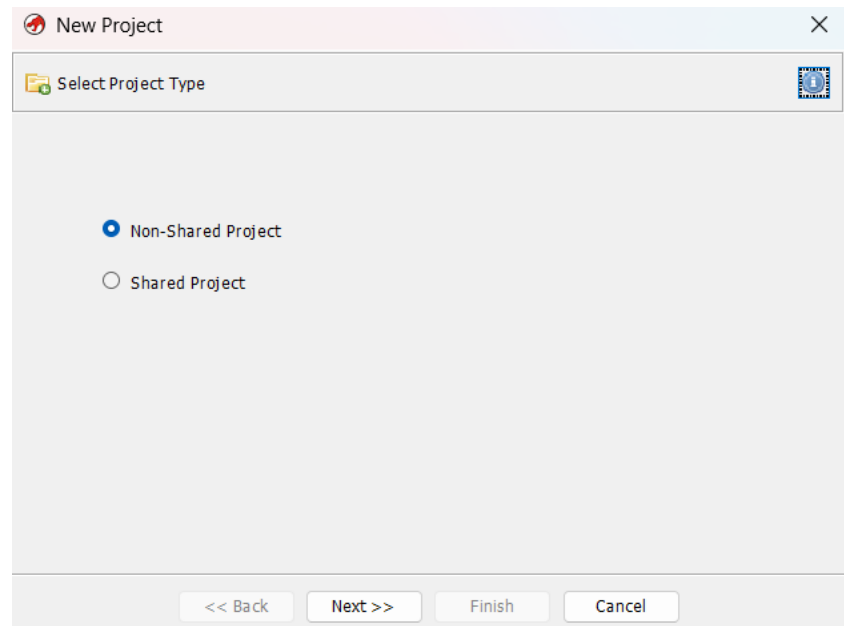
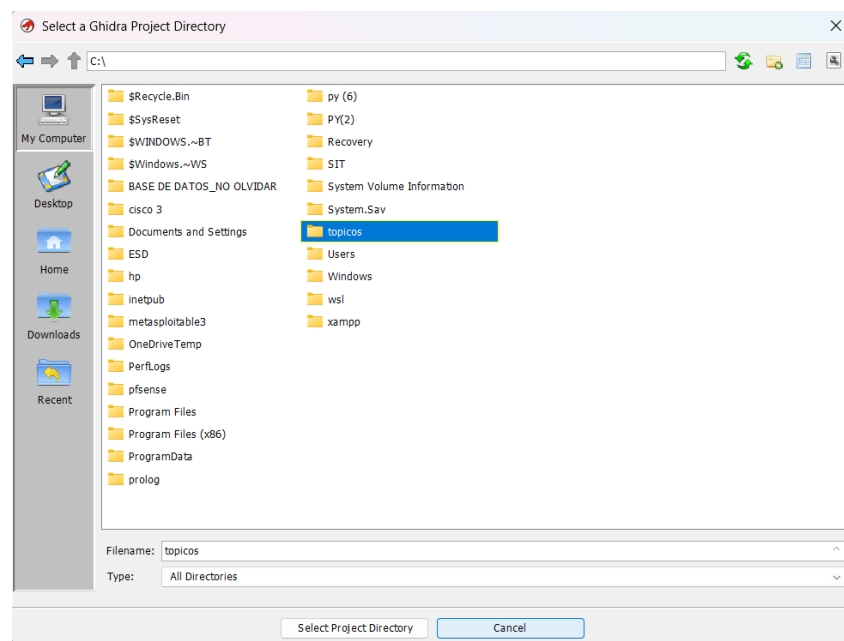


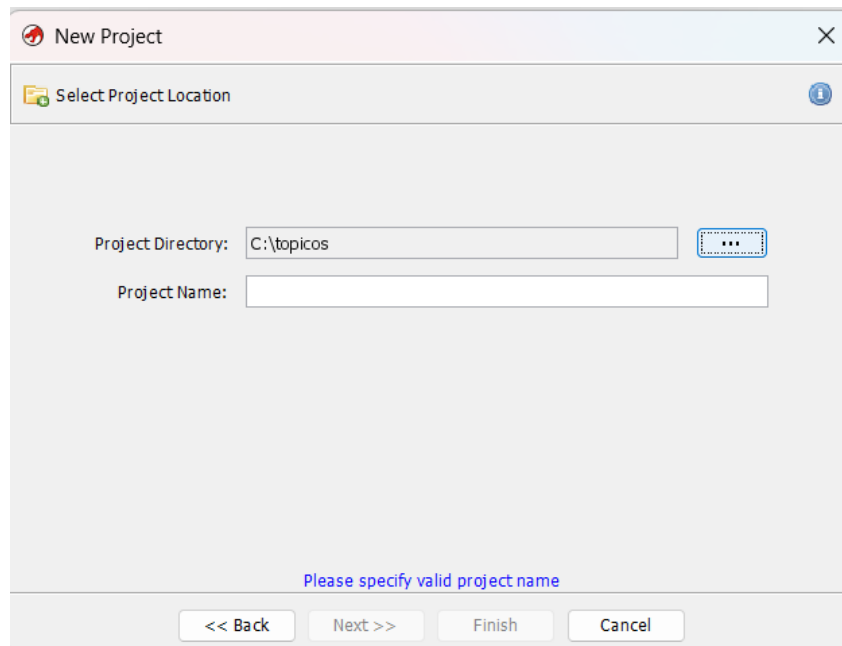
## 1. Creamos un proyecto nuevo

Abrimos Ghidra, fuimos a File > New Project, y elegimos Non-Shared Project.



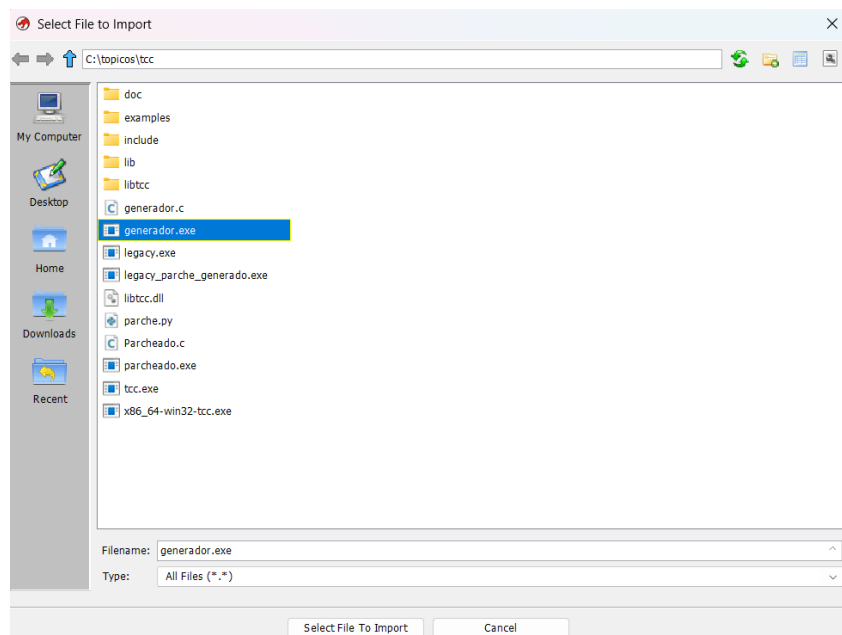
Elegimos la carpeta donde íbamos a guardar el proyecto:

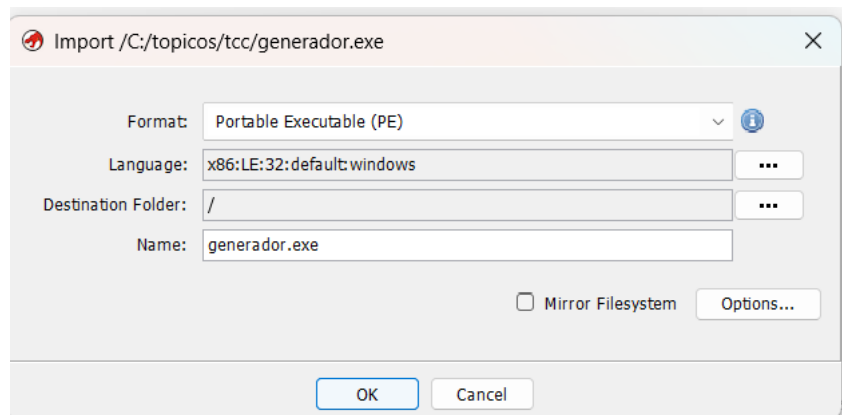




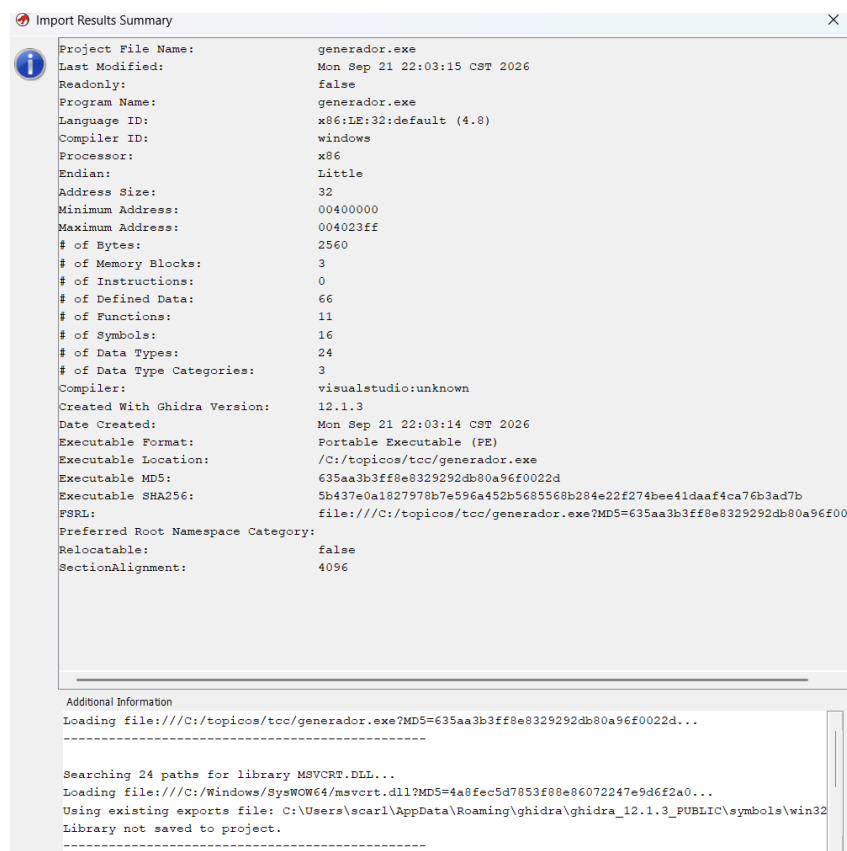
## 2. Importamos el ejecutable

Con el proyecto ya creado, fuimos a File > Import File y seleccionamos `generador.exe` (el que compilamos con TCC):





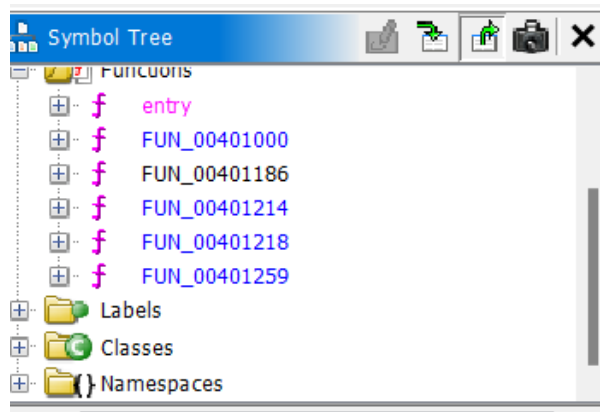
Al darle OK, Ghidra nos mostró el resumen de la importación (formato, cantidad de funciones detectadas, etc.):



### 3. Analizamos el ejecutable

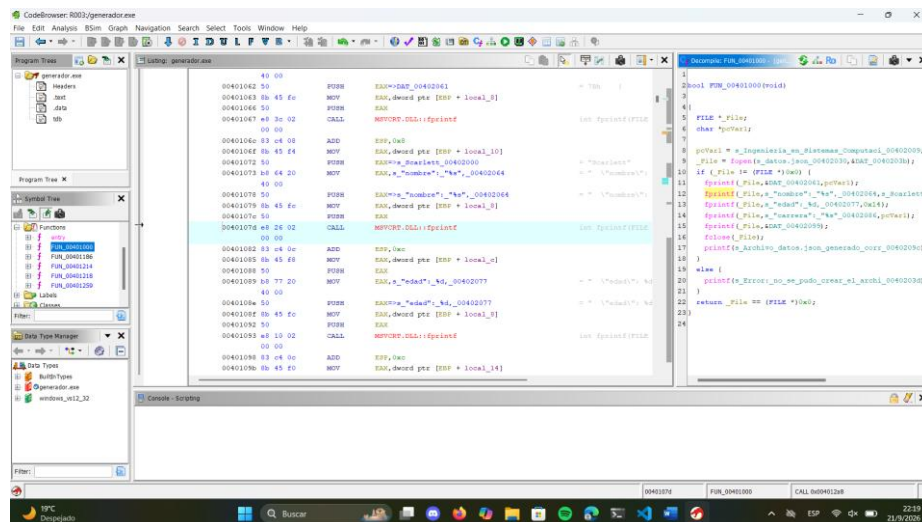
Le dimos doble clic a generador.exe dentro del proyecto para abrirlo con el CodeBrowser, y aceptamos que Ghidra lo analizara automáticamente (Yes > Analyze).

En la lista de funciones (Symbol Tree > Functions) aparecieron varias funciones. La mayoría son del arranque del programa (entry, FUN\_00401186, etc), pero identificamos FUN\_00401000 como la función principal, porque desde entry se ve que la llama directamente.



#### 4. Encontramos la lógica real (pseudocódigo)

Al abrir FUN\_00401000, en el panel de Decompile Ghidra ya nos mostró el pseudocódigo en C de la función, y en el panel de la izquierda (ensamblador) se alcanzan a ver los strings reales que usa el programa (nombre, edad, carrera, los mensajes de error, etc):



Con esto pudimos reconocer exactamente qué hacía el programa: abre un archivo (fopen), si se pudo abrir escribe con fprintf el JSON con nombre, edad y carrera, cierra el archivo (fclose) y muestra un mensaje de éxito o de error.